

One platform. Every business function.

Symfonix Suite is an all-in-one modular business management platform that unifies CRM, HR, Finance, CMS, and core operational workflows into a single, high-performance ecosystem — eliminating the integration friction of fragmented SaaS tools.

Business modules

Unified database

Composable workflows

Enterprise-ready

Introduction & Vision

Symfonix Suite was built for organizations that have outgrown the patchwork of disconnected software — where sales lives in one tool, payroll in another, and inventory in a third, each with its own login, data model, and sync failures.

The name **Symfonix** reflects the product philosophy: a symphony of business technologies, orchestrated with intention. Rather than forcing enterprises to become systems integrators, Symfonix Suite provides a *core system engine* that dynamically loads independent, decoupled business modules based on operational needs — CRM today, finance from day one, and additional capabilities as the organization evolves.

Built on a modern Laravel foundation with a modular architecture ([nwidart/laravel-modules](#)), Symfonix Suite targets agencies, service companies, distributors, and mid-market B2B enterprises that need a unified operational backbone without sacrificing the flexibility to enable only the modules they require.



Unified Ecosystem

Replace fragmented third-party SaaS subscriptions with one platform sharing a single database foundation and consistent user experience.



Modular by Design

Activate CRM, HR, Finance, CMS, and other business modules independently. Each module is decoupled yet deeply integrated at the data layer.



High Performance

Server-rendered admin with Inertia.js SPA navigation, optimized queries, and optional Redis caching for production workloads.



Global Ready

Built-in localization (English, Arabic, Turkish, and German), role-based access control, and multi-branch organizational structures.

Why Symfonix Suite?

Most B2B organizations accumulate software debt over years of growth. A CRM here, an accounting package there, a spreadsheet for inventory, and email for everything else. Each addition creates:

- **Data silos** — customer records in CRM don't match invoice recipients in Finance.
- **Integration tax** — Zapier workflows, CSV exports, and manual reconciliation consume engineering and operations time.
- **Permission sprawl** — access control fragmented across a dozen vendor admin panels.
- **Reporting blindness** — leadership cannot get a real-time, cross-functional view of the business.

Symfonix Suite eliminates this friction by design. When a deal closes in CRM, the project module can spin up delivery work. When a product sale is recorded, Finance captures the revenue. When HR onboards an employee, permissions propagate through Spatie's unified role system. One login. One truth.

Why companies choose Symfonix Suite



Lower software costs

Replace overlapping subscriptions and reduce the maintenance burden of third-party integrations.



Faster daily operations

Give sales, finance, projects, HR, and content teams connected workflows instead of repetitive manual handoffs.



Reliable business data

Keep customers, transactions, projects, and permissions in one governed system for clearer reporting and decisions.



Control and data ownership

Self-host the platform, define access by role, and retain control over company and customer information.



Growth without replacement

Start with the modules needed today and activate more capabilities as teams, branches, and processes expand.

Architectural Value Proposition for B2B Enterprises

For enterprise buyers and technical decision-makers, Symfonix Suite delivers value across three dimensions:



Total Cost of Ownership

Consolidate vendor contracts, reduce integration maintenance, and deploy on your own infrastructure or managed hosting — full data sovereignty.



Operational Velocity

Cross-module workflows execute in milliseconds against a shared schema. No webhook delays, no sync queues, no "data not yet available" dashboards.



Future-Proof Modularity

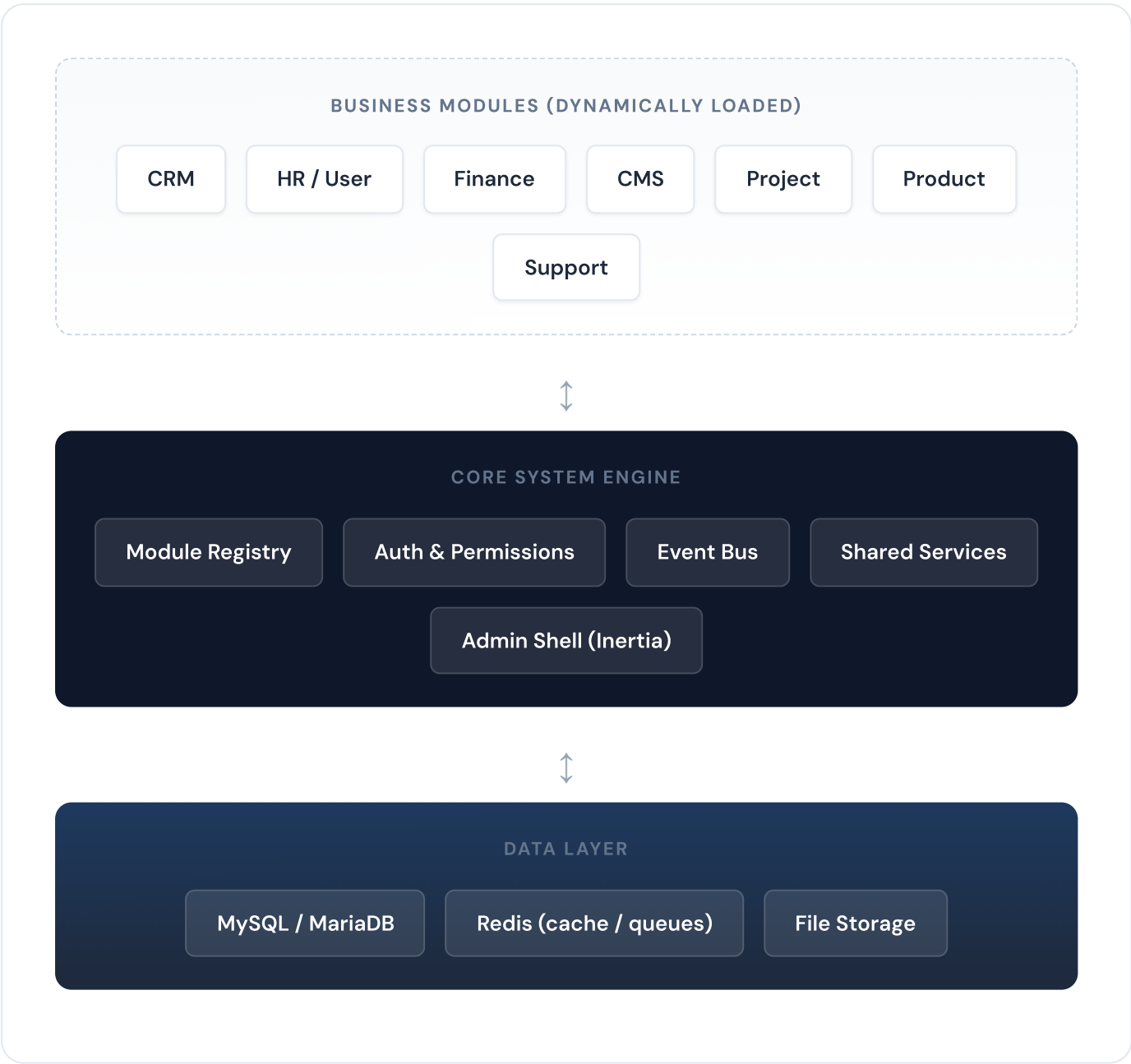
New business capabilities ship as installable modules without rewriting the core. The platform grows with the enterprise, not against it.



Designed for composability. Symfonix Suite is not an monolithic ERP you must adopt wholesale. Enable the modules your organization needs today and add others as requirements evolve — the core engine handles discovery, routing, and permission registration automatically.

System Architecture

Symfonix Suite follows a layered modular architecture where business logic lives in self-contained modules, orchestrated by a shared core engine and backed by a single relational database.



Each module is a first-class Laravel package under `Modules/` with its own routes, controllers, models, migrations, views, and service providers. The core engine discovers enabled modules at boot time, registers their service providers, merges route files, and runs migrations in dependency order.

The Core Engine

The **Core** and **Base** modules form the system engine. Core provides the installation command (`php artisan app:install`), shared helpers, and bootstrap logic. Base handles global settings, countries, branches, SEO configuration, and the file manager integration.

Key responsibilities of the core engine include:

- 1. **Module lifecycle** — register, enable, disable, and migrate modules through standardized artisan commands.
- 2. **Permission synthesis** — aggregate permission definitions from all active modules into a unified Spatie Permission registry.
- 3. **Admin shell** — provide the Metronic-themed admin layout, navigation scaffolding, and Inertia page rendering pipeline.
- 4. **Cross-module events** — Laravel's event system enables loose coupling (e.g., Finance listens for invoice-sent events from CRM subscriptions).

Data Consistency

Because all modules share a single database, referential integrity is enforced at the schema level — not approximated through nightly sync jobs. Foreign keys connect customers to deals, employees to salary records, products to stock movements, and projects to companies.

Spatie Laravel Data DTOs standardize data transfer between layers, ensuring type-safe payloads across module boundaries. Translatable models (via `spatie/laravel-translatable`) keep CMS content and product catalogs consistent across locales without duplicate records.

CONCERN	APPROACH	BENEFIT
Schema ownership	Per-module migrations	Modules evolve independently without central bottleneck
Shared entities	Core/Base models (User, Country, Branch)	Single source of truth for organizational data
Transactions	Laravel DB transactions	Atomic cross-table operations (e.g., stock deduction + invoice)
Audit trail	Database logging + Telescope (dev)	Traceability for compliance and debugging

Performance

Symfonix Suite is engineered for responsive admin experiences under real enterprise load:

- **Inertia.js** delivers SPA-like navigation without the complexity of a separate frontend API layer.

- **Eager loading & repository patterns** in high-traffic modules (CRM, Finance) prevent N+1 query problems.
- **Redis** (optional) accelerates session storage, cache, and queue processing for background jobs like invoice PDF generation and notification dispatch.
- **Laravel Pulse** provides production performance monitoring when enabled.
- **Asset pipeline** via Vite ensures optimized, cache-busted frontend bundles.

Core Modules & Interfaces

Symfonix Suite ships with a comprehensive set of business modules. Each module exposes a dedicated admin interface, API routes where applicable, and integrates with the shared permission and notification systems.

MODULE	DOMAIN	KEY CAPABILITIES
CRM	Sales & relationships	Leads, deals, companies, pipeline stages, subscriptions, activities, marketing
User	HR & access	Users, employees, roles, permissions, portal dashboard
Finance	Accounting	Multi-currency transactions, salaries, commissions, expenses, journal entries, invoices
Cms	Content	Pages, blog posts, FAQs, translatable content
Product	Catalog	Product categories, catalog items, sales integration
Project	Delivery	Projects, statuses, use cases, client delivery tracking
Support	Customer care	Tickets, contact subscribers, complaints

CRM & Pipelines

The CRM module is the revenue engine of Symfonix Suite. It manages the full lifecycle from first contact to closed deal, with visual pipeline stages, activity tracking, and subscription management for recurring revenue models.

Key Interfaces

- **Lead Management** — capture, qualify, and convert inbound leads with custom fields and source attribution.
- **Deal Pipeline** — Kanban-style pipeline with configurable stages (default: Lead → Qualified → Proposal → Negotiation → Closed Won/Lost). Drag-and-drop stage transitions with activity logging.
- **Company Records** — B2B account management with linked contacts, deals, projects, and subscription history.
- **Activities & Tasks** — schedule calls, meetings, and follow-ups tied to leads, deals, or companies.
- **Subscriptions** — manage recurring service contracts with renewal tracking and Finance integration for invoicing.
- **Marketing** — campaign tracking and contact form management for inbound lead generation.
- **Sales Targets** — set and monitor team and individual sales quotas against pipeline performance.

Pipeline Workflow

Deals progress through pipeline stages defined in `CRM/database/seaders`. Stage transitions trigger events that other modules can listen to — for example, moving a deal to "Closed Won" can automatically create a Project record and generate a Finance transaction.



CRM permissions are granular: `CRM Management` for full access and `CRM View All` for read-only visibility across all pipelines — useful for executives and cross-functional teams.

Human Resources

The User module serves as Symfonix Suite's HR and identity backbone. It bridges employee records with system authentication, role assignment, and payroll integration through the Finance module.

Key Interfaces

- **Employee Profiles** — comprehensive employee records linked to user accounts, including contact details, department assignment, and employment status.
- **Role-Based Access Control** — powered by Spatie Laravel Permission, with predefined roles (Admin) and custom role creation in the admin panel.
- **Permission Matrix** — fine-grained permissions per module (CRM Management, Hr Management, Finance Management, etc.) assignable to any role.
- **Portal Dashboard** — employee-facing portal for self-service access to assigned resources and personal dashboards.
- **Team Module Integration** — public-facing team member profiles for the company website, managed alongside internal HR records.

HR ↔ Finance Bridge

Employee records connect directly to Finance salary and commission tracking. When payroll runs, transactions are attributed to the correct employee with full audit history — no CSV imports between HR and accounting systems.

Finance

The Finance module provides end-to-end, multi-currency financial operations — from daily transaction logging to salary disbursement, commission calculation, invoicing, and expense categorization. It listens to events from CRM and Product modules to maintain accurate books without manual data entry.

Key Interfaces

- **Transactions** — real-time ledger of income and expenses with categorization, filtering, and today's-transactions dashboard widget (Livewire).
- **Journal Entries** — double-entry bookkeeping support for organizations requiring formal accounting practices.
- **Salaries** — recurring payroll management tied to employee records in the User module.
- **Commissions** — sales commission tracking linked to CRM deal closures and sales targets.
- **Expenses** — operational expense logging with customizable expense categories.
- **Product Sales** — revenue recording from Product catalog sales, connected to the broader commercial workflow for accurate reporting.
- **Invoice Notifications** — automated customer notifications when invoices are sent, via Laravel event listeners.
- **Currency Context** — separate accounting-base, transaction, and per-session display currencies with posting-time exchange-rate snapshots.

Event-Driven Integration

Finance uses Laravel's event/listener pattern for cross-module workflows. For example, `NotifyCustomerOfInvoiceSent` listens for invoice events and dispatches notifications — keeping Finance decoupled from CRM implementation details while maintaining data flow.

Multi-currency Setup

Symfonix separates three currency concepts: the **default currency** is the accounting base, each business record retains its **transaction currency**, and the admin-header selector controls only the user's **display currency**. Journal entries snapshot both the posting-time source-to-base rate and converted base amount; changing display currency never changes the stored posting.

SETTING	PURPOSE	DEFAULT
<code>FINANCE_DEFAULT_CURRENCY</code>	Accounting base currency	<code>USD</code>
<code>FINANCE_SUPPORTED_CURRENCIES</code>	Comma-separated currencies available to users	<code>USD, EUR, GBP, TRY</code>
<code>FIXER_API_KEY</code>	Fixer or API Layer credential	Empty
<code>FIXER_BASE_URL</code>	Exchange-rate API endpoint	<code>https://data.fixer.io/api</code>

Users with `Settings Management` can override the default currency and Fixer key under **System Configurations** → **Finance**; stored settings take precedence over environment values. The supported-currency list remains environment/config driven. Free Fixer plans return EUR-based rates, so Symfonix derives non-EUR defaults through EUR cross-rates.

TERMINAL

```
# Queue a rate update (requires a running queue worker)
php artisan finance:fetch-exchange-rates

# Fetch immediately, optionally overriding the base
php artisan finance:fetch-exchange-rates --sync
php artisan finance:fetch-exchange-rates --base=EUR --sync
```



Do not use fallback rates for production accounting. Installation seeds approximate baseline rates, and a missing Fixer key or unavailable stored rate can result in a `1.0` fallback. Configure and monitor synchronization before posting live transactions. Changing the default currency after postings exist requires a controlled migration because historical base amounts retain the original accounting base.

Content Management (CMS)

The CMS module powers the public-facing website alongside the admin panel. It provides a full content authoring environment with multilingual support, SEO integration, and rich text editing via TinyMCE.

Key Interfaces

- **Pages** — create and manage static pages (About, Services, Contact) with slug-based routing and translatable content fields.
- **Blog** — publish articles with categories, read-time estimation, featured images, and SEO metadata.
- **FAQs** — structured frequently-asked-questions management for support and marketing pages.
- **SEO Settings** — global and per-page SEO configuration managed through the Base module's SEO interface, including meta titles, descriptions, and Open Graph tags.
- **Services Showcase** — the Services module extends CMS capabilities with service categories and offering pages for agencies and consultancies.

Localization

Content is translatable across English, Arabic, Turkish, and German via [mcamara/laravel-localization](#) and Spatie Translatable. RTL layout support is built into the public site theme for Arabic locales.

Getting Started

This guide walks through deploying Symfonix Suite from a fresh clone to a running admin panel with all core modules installed. The platform uses Laravel 13 with modular architecture — each business module is a self-contained package activated through the module system.

Requirements

Ensure your server or local development environment meets these minimum specifications:

DEPENDENCY	VERSION	NOTES
PHP	8.3+	Extensions: BCMath, CType, JSON, Mbstring, OpenSSL, PDO, Tokenizer, XML
Composer	2.x	PHP dependency management
Node.js	18+	With npm for frontend asset compilation
MySQL / MariaDB	8.0+ / 10.6+	Primary data store — single database for all modules
Redis	6+	Optional — recommended for production cache and queues

Installation

Follow these steps to deploy a fresh Symfonix Suite instance:

1

Clone and configure environment

Copy the environment file and set your database credentials:

TERMINAL

```
cp .env.example .env
```

Edit `.env` with your database connection:

```
.ENV
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=symfonix
DB_USERNAME=root
DB_PASSWORD=

FINANCE_DEFAULT_CURRENCY=USD
FINANCE_SUPPORTED_CURRENCIES=USD,EUR,GBP,TRY
FIXER_API_KEY=
FIXER_BASE_URL=https://data.fixer.io/api
```

2

Install dependencies

Install PHP and Node.js dependencies, then compile frontend assets:

TERMINAL

```
composer install
npm install && npm run build
```

3

Run the install command

The `app:install` artisan command handles the full bootstrap — migrations, seed data, permissions, CRM pipeline stages, baseline currency settings and rates, and admin user creation:

TERMINAL

```
php artisan app:install
```

Custom admin credentials:

TERMINAL

```
php artisan app:install \  
  --email=you@company.com \  
  --password=secret \  
  --name="Your Name" \  
  --mobile=1234567890
```

4

Start the application

TERMINAL

```
php artisan serve
```

Visit `/admin` and sign in with the credentials displayed after installation.



The install command generates `APP_KEY`, runs all module migrations, seeds countries and baseline currency rates, creates application permissions, seeds default CRM pipeline stages, and assigns the Admin role with full permissions.

Module Deployment

Symfonix Suite modules follow the [nwidart/laravel-modules](#) convention. Each module lives in `Modules/{ModuleName}/` with a standard structure:

MODULE STRUCTURE

```
Modules/
├─ CRM/
│   ├─ app/
│   │   ├─ Http/Controllers/
│   │   ├─ Models/
│   │   ├─ Actions/
│   │   └─ Providers/
│   ├─ database/migrations/
│   ├─ resources/views/
│   ├─ routes/
│   │   ├─ web.php
│   │   └─ api.php
│   ├─ module.json
│   └─ composer.json
├─ Finance/
├─ Product/
└─ ...
```

Enabling & Disabling Modules

Module activation is controlled via `module.json` and the module status file:

TERMINAL

```
# List all modules and their status
php artisan module:list

# Enable a module
php artisan module:enable Stock

# Disable a module
php artisan module:disable Testimonial

# Run migrations for a specific module
php artisan module:migrate CRM
```

Creating a New Module

To scaffold a new business module (e.g., a custom reporting module):

TERMINAL

```
php artisan module:make Reporting

# Generate module components
php artisan module:make-controller ReportController Reporting
php artisan module:make-model Report Reporting -m
php artisan module:make-migration create_reports_table Reporting
```

Register permissions in your module's service provider and add them to the install command's permission seeder for automatic provisioning on fresh installs.

Frontend Development

Run the Vite dev server alongside the Laravel application for hot module replacement during development:

TERMINAL

```
# Terminal 1 - Laravel
php artisan serve

# Terminal 2 - Vite HMR
npm run dev

# Terminal 3 - Queue worker (for async jobs)
php artisan queue:work
```

Permissions & Roles

Symfonix Suite uses [Spatie Laravel Permission](#) for authorization. Permissions are created during `app:install` and assigned to roles through the admin panel.

PERMISSION	MODULE	SCOPE
Settings Management	Base	Global settings, branches, SEO
CMS Management	Cms	Pages, blog, FAQs
Hr Management	User	Employees, user accounts
CRM Management	CRM	Full CRM read/write access
CRM View All	CRM	Read-only across all pipelines
Finance Management	Finance	Transactions, salaries, expenses
Project Management	Project	Projects and delivery tracking
Product Management	Product	Catalog and categories
Services Management	Services	Service offerings
Support Management	Support	Tickets and subscribers
Sales Management	CRM / Product	Sales operations
App Monitoring	Core	Telescope, Pulse access
Logs Management	Base	Application log viewer

Assign permissions to roles in the admin panel under **User Management** → **Roles**. The default **Admin** role receives all permissions on fresh install.

Production Deployment

For production environments, apply these optimizations after installation:

1

Configure environment

```
.ENV
```

```
APP_ENV=production
APP_DEBUG=false
CACHE_DRIVER=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis
```

2

Install production dependencies

TERMINAL

```
composer install --no-dev --optimize-autoloader
npm ci && npm run build
```

3

Run migrations and cache

TERMINAL

```
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

4

Configure background services

Set up a queue worker and scheduler via cron. The scheduler performs a synchronous exchange-rate refresh every hour; queued manual fetches require the worker.

CRONTAB

```
* * * * * cd /path-to-symfonix && php artisan schedule:run >> /dev/null 2>&1
```



Production checklist: Ensure `APP_DEBUG=false` , restrict `/admin` access if needed, enable two-factor authentication, never commit `.env` , configure a valid Fixer key, and monitor the last successful rate fetch under **System Configurations** → **Finance**. Stored rates have no automatic staleness rejection.